

TECHNOLOGY & PLATFORM INVENTORY

Ruhvi Technology Stack

Comprehensive inventory of all tools, platforms, APIs, and technologies currently integrated into the Ruhvi e-commerce platform, with detailed descriptions of each component's purpose within the workflow.

Project: Ruhvi E-Commerce (Next.js 15, App Router)

Total Technologies: 70+ • **Revision:** 30 August 2026 (reflects current codebase)

Table of Contents

- [1. Frontend Framework & Rendering](#)
- [2. Hosting & Infrastructure](#)
- [3. Database & Storage](#)
- [4. Authentication & Authorization](#)
- [5. AI & Machine Learning](#)
- [6. Analytics, Monitoring & Error Tracking](#)
- [7. Push Notifications](#)
- [8. Email Services](#)
- [9. Payments](#)
- [10. Image, Media & 3D](#)

- 11. [Shipping & Logistics](#)
- 12. [CRM & Customer Support](#)
- 13. [Messaging & Communication](#)
- 14. [Forms, Validation & Security](#)
- 15. [UI Components & Design System](#)
- 16. [Development Tooling & CI/CD](#)
- 17. [Mobile & Cross-Platform](#)
- 18. [Other Integrations](#)

1. Frontend Framework & Rendering

Technology	Category	Purpose & Workflow Role
Next.js 15	Framework	React-based full-stack framework with App Router. Provides server-side rendering, static generation, incremental static regeneration (ISR), API routes, middleware, and streaming. The core application shell that handles routing, data fetching, and page rendering for all customer-facing and admin pages.
React 19	Library	UI component library. Powers all interactive elements — context providers (Auth, Cart, Wishlist), client components (Navbar, Footer, ProductCard, SearchBar, Chat), and hooks for state management and side effects.
TypeScript	Language	Statically-typed superset of JavaScript. Used throughout the codebase for type safety, interface definitions (<code>Product</code> , <code>Category</code> , <code>UserProfile</code> , API request/response shapes), and improved developer experience with autocompletion and compile-time error detection.
Tailwind CSS 3	Framework	Utility-first CSS framework. Provides the design system with custom color tokens (gold, cream, charcoal, midnight, rasa, champagne, taupe) and responsive utilities. Used for all component styling via inline utility classes, with dark mode via <code>class</code> strategy.
PostCSS	Tool	CSS transformation pipeline. Compiles Tailwind CSS directives and applies Autoprefixer during the build step. Configured via <code>postcss.config.js</code> .
Autoprefixer	Plugin	PostCSS plugin that automatically adds vendor prefixes to CSS rules for cross-browser compatibility. Runs as part of the build pipeline.

2. Hosting & Infrastructure

Technology	Category	Purpose & Workflow Role
Vercel	Hosting	Deployment and hosting platform for the Next.js application. Provides global edge network, serverless functions for API routes, automatic HTTPS/SSL, and CI/CD integration via Git. The primary production environment for all customer-facing and admin interfaces.
Vercel SpeedInsights	Monitoring	Real-user performance monitoring integrated into the Vercel dashboard. Tracks Core Web Vitals (LCP, CLS, INP) for every page and provides trend analysis. Rendered globally via <code>@vercel/speed-insights/next</code> in the root layout.
Vercel Cron Jobs	Scheduler	Scheduled serverless function invocations. Configured via <code>vercel.json</code> to trigger <code>/api/cron/automations</code> daily at 10:00 IST for automated background tasks.
Docker	Containerization	Used to deploy the self-hosted EspoCRM instance on a VPS. The <code>deploy/espocrm/</code> directory contains a <code>docker-compose.yml</code> and <code>Caddyfile</code> for orchestrating the EspoCRM service, database, and reverse proxy.

3. Database & Storage

Technology	Category	Purpose & Workflow Role
Supabase (PostgreSQL)	Database	Primary relational database. Stores all application data — users, products, categories, collections, orders, support tickets, wallet transactions, audit logs, AI provider credentials, model health, and settings. Accessed via the Supabase JS client on both server and client, with Row-Level Security (RLS) policies for data access control.
Supabase Storage	File Storage	Cloud object storage for user-uploaded files (product images, support attachments). Configured in <code>next.config.js</code> as a remote image pattern (<code>igrkrkxdantrolbldapj.supabase.co</code>).
Upstash Redis	Cache	Serverless Redis instance for caching and rate limiting. Provides <code>cacheGet</code> / <code>cacheSet</code> / <code>cacheWrap</code> helpers for data caching, <code>rateLimit</code> / <code>rateLimitSliding</code> for API rate limiting via Redis scripts, and a read-only replica for scaling. Used to reduce database load and protect API endpoints from abuse.
Supabase Realtime	Real-time	WebSocket-based real-time subscriptions. Used in <code>AuthContext</code> to subscribe to <code>postgres_changes</code> on the <code>users</code> table, enabling live wallet balance and profile updates without polling.

4. Authentication & Authorization

Technology	Category	Purpose & Workflow Role
Firebase Authentication	Auth	Primary authentication provider for customer-facing accounts. Handles email/password, phone (OTP), and Google SSO logins. The <code>onAuthStateChanged</code> listener in <code>AuthContext</code> manages the session lifecycle. Firebase ID tokens are exchanged for custom Supabase JWTs to enable database access.
Supabase Auth (Custom Token)	Auth	Secondary auth layer. The Supabase client uses an <code>accessToken</code> callback that fetches a custom JWT minted server-side from the Firebase ID token. This bridges Firebase authentication with Supabase's RLS policies, allowing the <code>anon</code> key to be used with user-scoped permissions.
jose	Library	JWT signing and verification library. Used for: (a) signing session cookies (<code>__session</code>) for portal/admin auth, (b) verifying session tokens in middleware for RBAC, (c) signing Google service account JWTs for Firebase Admin REST API calls, (d) decoding custom Supabase tokens in the client.
Supabase SSR (<code>@supabase/ssr</code>)	Library	Server-side rendering helpers for Supabase. Provides <code>createServerClient</code> used in middleware, server components, and route handlers to create authenticated Supabase clients with cookie-based session management.

5. AI & Machine Learning

Technology	Category	Purpose & Workflow Role
Google Generative AI (Gemini)	AI Provider	AI model provider for the Ruhvi AI orchestration engine. Powers the customer support chatbot ("Gia"), product description generation, AI playground, and admin AI features. Accessed via <code>@google/generative-ai</code> SDK through the unified provider abstraction in <code>src/lib/ai/providers/gemini.ts</code> .
OpenAI	AI Provider	AI model provider integrated into the multi-provider routing engine. Supports GPT models for AI generation tasks. Credentials are managed via the <code>ai_provider_credentials</code> database table with health tracking and failover.
Anthropic (Claude)	AI Provider	AI model provider for Claude models. Integrated through the same unified provider interface, with support for credential rotation, error classification, and automatic failover across providers.
DeepSeek	AI Provider	AI model provider for DeepSeek models. Part of the multi-provider routing strategy, enabling cost optimization and redundancy by distributing requests across providers based on priority and health status.
OpenRouter	AI Gateway	AI model aggregation gateway. Provides a unified API endpoint to access multiple models from different providers with a single integration point. Used as an alternative routing path within the AI orchestration engine.

Technology	Category	Purpose & Workflow Role
Custom AI Provider Framework	Framework	Extensible provider abstraction (<code>src/lib/ai/providers/custom.ts</code>) that allows arbitrary API-compatible AI providers to be added via admin configuration. Supports OpenAI-compatible API schemas, enabling future provider additions without code changes.
AI Orchestration Engine	System	Centralized AI request handler (<code>src/lib/ai/index.ts</code> , ~1100 lines) implementing two-level routing: provider selection (priority/round-robin/best-responsiveness) and credential selection within a provider. Features include error classification, atomic health state transitions, model-level fallback, infinite-loop prevention, structured observability, and rate-limit/quota-exhaustion handling. Recent refinement: 5xx server errors are classified before rate-limit patterns, and Gemini <code>RESOURCE_EXHAUSTED</code> maps to <code>QUOTA_EXHAUSTED</code> .

6. Analytics, Monitoring & Error Tracking

Technology	Category	Purpose & Workflow Role
PostHog	Analytics	Product analytics, session recording, and feature flags. Captures page views, custom events, and user sessions. The client-side SDK (<code>posthog-js</code>) is initialized in the root layout with session recording enabled (masking sensitive inputs). The server-side client (<code>posthog-node</code>) is used in API routes and for admin analytics queries. Includes a custom PII redaction layer to sanitize sensitive data before ingestion.
Google Analytics (GA4)	Analytics	Web analytics via the <code>gtag.js</code> library. Tracks page views, e-commerce events (add_to_cart, remove_from_cart, view_item, view_item_list, purchase, search), and custom events. The <code>GoogleAnalytics</code> component fires on every route change with path-based tracking.
Meta Pixel (Facebook)	Analytics	Conversion tracking and ad optimization pixel. Tracks PageView, AddToCart, Purchase, and other e-commerce events. Includes a server-side Conversions API (CAPI) via <code>/api/capi</code> for server-to-server event deduplication, improving ad attribution accuracy.
Microsoft Clarity	Analytics	Session recording and heatmap tool. Records user sessions (scroll, click, navigation) to identify usability issues and conversion friction. Loaded via <code>afterInteractive</code> script strategy in the root layout.
Sentry	Error Tracking	Error tracking and performance monitoring. Configured for server (<code>sentry.server.config.ts</code>), client (<code>sentry.client.config.ts</code>), and edge (<code>sentry.edge.config.ts</code>) runtimes with 100% traces sampling. Integrates with the Next.js build pipeline via <code>@sentry/nextjs</code> to upload source maps and enable automatic instrumentation of Vercel Cron Monitors.

7. Push Notifications

Technology	Category	Purpose & Workflow Role
Firebase Cloud Messaging (FCM)	Push	Push notification service for web and mobile. Implemented via <code>firebase/messaging</code> SDK. The <code>FcmInit</code> component requests permission and obtains FCM tokens for authenticated users, displays foreground notifications via toast, and integrates with a service worker (<code>public/firebase-messaging-sw.js</code>) for background notifications.
OneSignal	Push	Push notification service with cross-platform support. Initialized via <code>react-onesignal</code> SDK in the <code>OneSignalInit</code> component with a configurable notification button. Includes a service worker (<code>public/OneSignalSDKWorker.js</code>) for handling background push events.

8. Email Services

Technology	Category	Purpose & Workflow Role
Resend	Email	Transactional email service. Sends all customer-facing emails: welcome emails, order confirmations, shipping updates (shipped/out-for-delivery/delivered), order cancellations, payment received/failed notifications, password reset links, and support ticket updates (created/replied/resolved). Uses Handlebars-compiled templates from the <code>Resend_Templates/</code> directory.
Brevo (Sendinblue)	Email	Marketing email service. Sends promotional campaigns, newsletters, and marketing communications. Configured via API key with sender name/email. The <code>src/lib/brevo.ts</code> module provides SMTP email sending, and a separate MCP integration (<code>src/lib/brevo/mcp.ts</code>) enables AI-driven campaign management through the Model Context Protocol.
Handlebars	Template Engine	Server-side HTML template engine. Compiles email templates from <code>Resend_Templates/</code> with dynamic data (order details, customer info, ticket status) for transactional email rendering. Templates are loaded from <code>process.cwd()/Resend_Templates/</code> .

9. Payments

Technology	Category	Purpose & Workflow Role
PhonePe	Payment Gateway	Indian payment gateway handling UPI, credit/debit cards, and wallet payments. The <code>/api/checkout/phonepe</code> endpoint initiates payment requests, creates orders in the database, and returns a redirect URL. Webhooks at <code>/api/webhooks/phonepe</code> and <code>/api/webhooks/phonepe-</code>

Technology	Category	Purpose & Workflow Role
		<code>wallet</code> handle payment status callbacks to update order status and wallet top-ups.

10. Image, Media & 3D

Technology	Category	Purpose & Workflow Role
Cloudinary	Image CDN	Cloud-based image management and CDN. Handles image uploads (via unsigned presets), transformations (resize, crop, format conversion), and optimized delivery. The <code>cloudinaryService.ts</code> provides helper functions for upload, delete, and optimized URL generation with <code>f_auto,q_auto</code> and width transformations. Used for product images, gallery images, and homepage hero/banner imagery.
Next.js Image Component (<code>next/image</code>)	Optimization	Built-in image optimization component. Provides automatic lazy loading, responsive image <code>sizes</code> , format negotiation (WebP/AVIF), and placeholder support. Used throughout the app for product images, category thumbnails, hero images, logo, and UI icons. Configured with remote patterns for Supabase Storage, Cloudinary, and Unsplash.
PhotoSwipe + react-photoswipe-gallery	Lightbox	Touch-friendly image gallery lightbox. Used in <code>ProductImageGallery</code> for full-screen product image viewing with zoom, swipe navigation, and thumbnail previews. The gallery supports multiple images with hidden items for fullscreen swipe-through navigation.
Three.js	3D Engine	WebGL 3D rendering library. Used with <code>@react-three/fiber</code> (React renderer) and <code>@react-three/drei</code> (helpers) for 3D jewellery rendering. Components include <code>HeroRingScene</code> and <code>Hero3D</code> for the homepage hero, and <code>Product360Viewer</code> for interactive product viewing. The 360° viewer is a planned feature with placeholder UI.

11. Shipping & Logistics

Technology	Category	Purpose & Workflow Role
Shiprocket	Logistics	Indian shipping and fulfillment platform. The <code>src/lib/shiprocket.ts</code> module handles authentication (JWT token generation via email/password or API key), order creation, AWB (airway bill) assignment, and tracking. Webhooks at <code>/api/webhooks/shiprocket</code> process shipping status updates. The admin panel includes <code>/api/admin/shiprocket/create-order</code> for manual order creation.

12. CRM & Customer Support

Technology	Category	Purpose & Workflow Role
EspoCRM	CRM	Self-hosted CRM platform deployed via Docker Compose on a VPS at <code>crm.support.ruhvi.in</code> . Acts as the agent console for customer support, holding Cases (tickets) and Notes. Syncs bi-directionally with Supabase via the <code>src/lib/espo/</code> integration layer. Pulls customer context on demand via the Ruhvi Context API (<code>/api/integrations/espo/context</code>). Configured with custom entity definitions, hooks, and Ruhvi-specific UI templates.
EspoCRM Docker Deployment	Infrastructure	Docker Compose-based deployment of EspoCRM with Caddy reverse proxy. Includes installation scripts (<code>setup.sh</code> , <code>install-native.sh</code>), cron jobs for auto-closing stale tickets and purging old attachments, and a custom extension for Ruhvi-specific fields and automation hooks.
In-App AI Chatbot ("Gia")	Support	AI-powered customer support chatbot rendered globally. The <code>CustomerSupportChat</code> component provides a draggable, resizable chat window with a mascot ("Gia"), typewriter-style message rendering, quick-action buttons, and AI-powered responses via <code>/api/support/chat</code> . Can create support tickets automatically based on conversation context.

13. Messaging & Communication

Technology	Category	Purpose & Workflow Role
WhatsApp Business API (Meta)	Messaging	Customer messaging via WhatsApp. The <code>src/lib/whatsapp.ts</code> module sends template-based WhatsApp messages using the Meta Graph API (<code>graph.facebook.com/v19.0</code>). Supports order updates, shipping notifications, and promotional campaigns. The admin panel includes a WhatsApp campaign builder at <code>/api/admin/whatsapp/campaign</code> .

14. Forms, Validation & Security

Technology	Category	Purpose & Workflow Role
React Hook Form	Forms	Performant form library with uncontrolled inputs and reduced re-renders. Used for checkout forms, auth forms (login/signup), contact forms, newsletter signup, and admin data-entry forms. Integrated with Zod validation via <code>@hookform/resolvers</code> .
Zod	Validation	TypeScript-first schema validation library. Defines environment variable schemas (<code>src/lib/env.ts</code>), API request/response shapes, form validation rules, and database entity schemas. Provides type inference for end-to-end type safety.

Technology	Category	Purpose & Workflow Role
Cloudflare Turnstile	Security	Privacy-preserving CAPTCHA alternative. Integrated via <code>@marsidev/react-turnstile</code> React component. Used on login, signup, and contact forms to prevent bot submissions without user friction. Verified server-side at <code>/api/checkout/verify-turnstile</code> .
SSRF Protection	Security	Server-Side Request Forgery protection module (<code>src/lib/security/ssrf.ts</code>). Validates and sanitizes outbound URLs to prevent internal network probing from server-side requests, particularly in AI tool calls and webhook handlers.
Content Security & Security Headers	Security	Security headers applied via middleware: <code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: DENY</code> , <code>Strict-Transport-Security: max-age=31536000; includeSubDomains</code> (HSTS). <code>X-Robots-Tag: noindex, nofollow</code> applied to portal subdomains to prevent indexing of internal admin interfaces. Middleware now fails closed — internal-route auth failures redirect to login.

15. UI Components & Design System

Technology	Category	Purpose & Workflow Role
Lucide React	Icons	Tree-shakeable SVG icon library. Provides 40+ icon components used throughout the UI: navigation (ShoppingBag, Heart, User, Search, Wallet, Headphones), controls (X, Send, Filter, ArrowUpDown, RefreshCw, ZoomIn), and decorative elements (Sparkles, Gem, Award, Truck, ShieldCheck, Instagram, Facebook).
React Hot Toast	Notifications	Toast notification library. Used for user feedback across the app: cart additions/removals, wishlist toggles, auth errors, and FCM foreground notifications. Configured as a global <code>ToastProvider</code> in the root layout.
Recharts	Charts	Charting library for admin analytics dashboards. Used for sales reports, revenue trends, and support analytics visualizations. Provides bar, line, area, and pie chart components composable with React.
clsx	Utility	Conditional className construction utility. Used throughout components to dynamically combine Tailwind classes based on state, props, and conditions.
tailwind-merge	Utility	Tailwind class conflict resolution utility. Merges Tailwind classes intelligently, resolving conflicts between base and variant classes (e.g., merging <code>bg-white</code> with <code>bg-black</code>).
@tailwindcss/typography	Plugin	Tailwind plugin providing prose classes for rich text content. Used in blog posts, policy pages, and product descriptions for consistent typography styling of HTML/Markdown content.

Technology	Category	Purpose & Workflow Role
Custom Design System Components	Design System	Proprietary component library in <code>src/components/design-system/</code> : <code>GlassPanel</code> , <code>DepthCard</code> , <code>DepthButton</code> , <code>SpatialPage</code> , <code>FloatingParticles</code> , <code>GoldOrb</code> , <code>Carousel3D</code> , <code>BotMascot</code> . Provides the premium jewellery brand aesthetic with glassmorphism, depth effects, and spatial UI elements.

16. Development Tooling & CI/CD

Technology	Category	Purpose & Workflow Role
ESLint + eslint-config-next	Linting	Code linting configured with Next.js recommended rules. Enforces React best practices, accessibility, and import conventions. Runs via <code>npm run lint</code> and as a pre-commit hook via lint-staged.
Prettier + prettier-plugin-tailwindcss	Formatting	Code formatter with Tailwind class sorting plugin. Ensures consistent code style and Tailwind utility class ordering across the codebase. Configured in <code>.prettierrc</code> .
Jest + ts-jest	Testing	Unit test runner for TypeScript. Configured via <code>jest.config.js</code> with path aliasing (<code>@/</code> → <code>src/</code>), <code>ts-jest</code> transform for TypeScript compilation, and <code>node</code> test environment. Test files are discovered via <code>**/__tests__/**/*.test.ts</code> . Currently used for AI routing unit tests (with <code>next/headers</code> and <code>@supabase/ssr</code> mocked). Run via <code>npm test</code> or <code>npm run test:watch</code> .
Husky	Git Hooks	Git hooks manager. Triggers lint-staged on pre-commit to ensure code quality before commits are made. Configured via <code>prepare</code> script in <code>package.json</code> .
lint-staged	Git Hooks	Staged file linter. Runs ESLint and Prettier only on staged files during pre-commit, preventing full-codebase linting overhead. Configured in <code>.lintstagedrc.js</code> .
Sentry Webpack Plugin	Build	Webpack plugin that uploads source maps to Sentry during build. Configured in <code>next.config.js</code> via <code>withSentryConfig</code> , enabling precise error stack traces in production. Includes tree-shaking for Sentry debug logs and automatic Vercel Cron Monitor instrumentation.
Favicon Generator Script	Tooling	Custom Node.js script (<code>scripts/generate_favicons.js</code>) for generating favicon assets in various sizes and formats from a source image.

17. Mobile & Cross-Platform

Technology	Category	Purpose & Workflow Role
Capacitor	Mobile	Cross-platform mobile app framework from the Ionic team. Configured in <code>capacitor.config.json</code> with app ID <code>in.ruhvi.app</code> and name "Ruhvi Jewels". Wraps the Next.js web app as a native Android/iOS application with a SplashScreen plugin. The <code>webDir</code> is set to <code>out</code> for static export.

18. Other Integrations

Technology	Category	Purpose & Workflow Role
Model Context Protocol (MCP) SDK	Integration	Standardized protocol for AI-model-to-tool communication. The <code>@modelcontextprotocol/sdk</code> enables AI agents to interact with external tools and data sources. Integrated with Brevo for AI-driven email campaign management (<code>src/lib/brevo/mcp.ts</code>).
Supabase Supabase-js	Client	Official Supabase JavaScript client library. Used on both client (<code>createClient</code> with singleton pattern and custom <code>accessToken</code> callback) and server (<code>createServerClient</code> with cookie-based session). Provides database queries, real-time subscriptions, and storage access.
Firebase Admin REST API	Auth	Server-side Firebase user management via Google Identity Toolkit REST APIs. Implements user creation, password reset link generation, and user lookup using <code>jose</code> -signed service account JWTs. Avoids the <code>firebase-admin</code> npm package's ESM bundler conflicts on Vercel Serverless.
Service Workers	PWA	Two service workers in <code>public/</code> : <code>firebase-messaging-sw.js</code> for handling FCM background push notifications, and <code>OneSignalSDKWorker.js</code> for OneSignal push notification handling.
Web Manifest	PWA	<code>public/site.webmanifest</code> provides Progressive Web App metadata including app name, icons, theme color, and display mode for mobile home-screen installation.

Summary

The Ruhvi technology stack comprises **70+ distinct technologies** organized into 18 categories. The architecture follows a **Next.js 15 + Supabase** core with a **multi-provider AI orchestration engine**, **Firebase-first authentication** bridged to Supabase via custom JWT, **dual analytics and push notification systems** (each with overlapping functionality), and **self-hosted EspoCRM** for agent support workflows. The stack is deployed on **Vercel** with a **Docker-based EspoCRM** instance on a separate VPS. Unit testing is provided via **Jest + ts-jest**, added alongside the existing linting/formatting pipeline.